

Google Summer of Code, 2026

Proposal for CDLI

Project: [Revamping CDLI Mobile App \(React Native\)](#)



Name: Shiva Gupta

GitHub: [ShivaGupta-14](#)

GitLab: [ShivaGupta-14](#)

LinkedIn: [Shiva Gupta](#)

Email: shivascalerite@gmail.com

Time Zone: Indian Standard Time (IST), UTC +5:30

Project Size: Large

Mentors: Daksh Paleria, Nisheal John

TABLE OF CONTENTS

ABOUT ME.....3

PROJECT OVERVIEW.....4

WHY CDLI / WHY THIS PROJECT..... 5

WHY IT MATTERS TO CDLI.....6

BACKEND ANALYSIS..... 7

FRONTEND ARCHITECTURE & IMPLEMENTATION PLAN..... 13

MOBILE APP - FEATURE BREAKDOWN..... 18

UX FEATURE..... 22

PROJECT TIMELINE & AVAILABILITY..... 24

BUFFER STRATEGY & MILESTONES.....26

EXPECTATIONS FROM MENTORS..... 27

CONTRIBUTIONS SO FAR..... 28

WHY ME.....30

FUTURE WORK.....30

USE OF AI TOOLS..... 31

ABOUT ME

Hello :) My name is Shiva Gupta, a second-year Computer Science student at Scaler School of Technology, Bangalore. I enjoy building software and understanding how systems work behind the scenes. I primarily work with TypeScript, JavaScript, the MERN stack, and Java with Spring Boot, and I am interested in building practical applications that solve real problems.

I began contributing to open source in October 2025 to understand how real-world software development works beyond coursework. Since then, I have contributed to organizations including CCEXtractor, JSON Schema, and AOSSIE, gaining experience working in collaborative open-source environments and structured codebases.

I joined the CDLI project earlier this year and started contributing fixes and improvements to the framework, including resolving issues in the setup scripts that made installation easier for new contributors. I also help moderate the project's Slack channel, where I guide contributors in setting up the framework locally and assist with Docker setup, database configuration, and environment issues.

Outside of programming, I have always been curious about history and the evolution of civilizations, especially how societies recorded knowledge and shaped the world we live in today. Working on CDLI allows me to combine software development with a project that helps preserve and make ancient cuneiform records more accessible, which makes contributing to this project especially meaningful to me.

PROJECT OVERVIEW

CDLI previously maintained native mobile applications: an iOS app (2013) and an Android app (2017). The iOS app remains on the App Store but crashes on launch and has not been updated since 2015. The Android app has been delisted. A Flutter-based rebuild was developed during GSoC 2020; however, the repository has seen no maintenance since late 2020 and depends on outdated Dart SDK versions and deprecated packages. As of today, CDLI has no functional mobile presence.

At the same time, the backend infrastructure is fully operational. The CakePHP admin panel allows admins to manage tablet entries stored in the `cdli_tablet` MariaDB table, with a publicly accessible JSON feed available at `/cdli-tablet/feed`. The endpoint requires no authentication, and is ready to be consumed by a client application.

This project proposes building a modern React Native application targeting both Android and iOS from a single codebase. The application will integrate directly with the existing backend and:

- Fetch tablet metadata and image references from the JSON feed and Highlights from the postings endpoint
- Display curated tablet collections with high-resolution imagery
- Support search and filtering
- Enable swipe-based navigation between tablets
- Implement pinch-to-zoom for tablet imagery
- Allow sharing of tablet records
- Provide offline caching for improved performance and accessibility

Because the backend content pipeline is already live, any changes made by curators in the admin panel will be immediately reflected in the mobile application without requiring redeployment.

The outcome will be a maintainable, production-ready mobile client built on a modern stack and aligned with CDLI's existing infrastructure.

WHY CDLI / WHY THIS PROJECT

What draws me to CDLI is not only the technical challenge, but the significance of the data it preserves. The cuneiform tablets archived by CDLI represent some of the earliest recorded systems of writing and administration in human history.

I have long been interested in history and the evolution of civilizations, particularly how societies documented knowledge and built systems of communication. Contributing to CDLI allows me to combine that interest with software engineering by building tools that make these ancient records more accessible to researchers, students, and the broader public.

This project is meaningful to me because it connects modern mobile technology with heritage preservation. Developing a sustainable, maintainable mobile client for CDLI is an opportunity to improve accessibility while supporting the long-term visibility of one of the world's most important cuneiform archives.

WHY IT MATTERS TO CDLI

Currently, CDLI does not have any active mobile application. The previous apps are either crashing or have been delisted from their platforms. Even the Flutter app developed earlier was not maintained after its initial development. Based on App Store reviews, some users appreciated having a mobile app, but complained about crashes and ended up using the website instead. There is clearly interest in a working mobile experience.

Earlier, CDLI also had to manage separate codebases for different platforms, which increased maintenance overhead. With a single React Native codebase, this overhead will be reduced significantly. This project will help CDLI regain a stable presence on mobile and tablets through a dedicated app, not just the website.

Community Survey (10 responses): Browse and Search were the most requested features (80% must-have), followed by Highlights (70%) and high-resolution image viewing (60%). Primary use cases include browsing tablet articles (80%), reading inscriptions (70%), and academic research (70%). Bookmarks and offline access were rated as “nice to have”. Platform preference indicates usage across both iOS and Android, reinforcing the need for a cross-platform solution. One suggestion included private notes on tablets, which can be explored as a future enhancement.

The proposed feature set is directly aligned with the highest-priority needs identified in the survey.

BACKEND ANALYSIS

1. Daily Tablet Feed:

- **Admin Panel:** <https://cdli.earth/admin/cdli-tablet/> (requires role 1, admin login)

The framework already includes a fully functional admin panel for managing tablet content.

Project Goal	Status	Implementation
Managing tablet entries (CRUD)	Working	add(), edit(), delete(), deleteAll() actions
Uploading and displaying artifact images	Working	File upload + served from /pubs/daily_tablets/
Editing metadata, descriptions, and publish dates	Working	All fields editable (theme, titles, descriptions, date, author)
Input validation and formatting	Working	type validation and FK constraint
Listing entries with pagination and search	Working	30 per page, search by theme or date
Ordering content by date for mobile delivery	Working	date_display DESC (Orderly behavior)
Returning structured JSON responses	Working	/cdli-tablet/feed JSON endpoint
Role-based access for tablet management	Partially implemented	Role 8 exists in the database, but access control is not yet enforced in RequestAccessComponent.php

Access Control: Currently restricted to Role 1 (Admin). Role 8 (tablet manager) already exists in the database but is not yet enforced. I will submit a MR to enable this role, allowing designated users to manage tablet entries without requiring full admin access.

- **Public API Endpoint :** Daily Tablet Feed - LIVE AND WORKING
Endpoint: GET <https://cdli.earth/cdli-tablet/feed>

Dedicated JSON feed built for mobile consumption. Requires no authentication.

Query Parameter	Endpoint Example	Behavior
?all=true	/feed?all=true	Returns all entries
?todaysdate=YYYY-MM-DD	/feed?todaysdate=2026-02-28	Returns 10 entries on or before the given date
?specificdate=YYYY-MM-DD	/feed?specificdate=2026-01-15	Returns 10 entries on the exact given date
?thumbnaildate	/feed?thumbnaildate	Returns entries on or before today (thumbnail mode)
(none)	/feed	Returns 10 most recent entries on or before today

- **Current Response (per entry):**

```
{
  "date": "2024-08-17",
  "thumbnail-url":
    "https://cdli.earth/pubs/daily_tablets/nabopolassar'sinscriptions03.jpg",
  "url":
    "https://cdli.earth/pubs/daily_tablets/nabopolassar'sinscriptions03.jpg",
  "blurb-title": "Nabopolassar's Inscriptions: 3",
  "theme": "Nabopolassar's Inscriptions",
  "blurb": "Clay barrel-cylinder with an inscription of the Neo-Babylonian king
    Nabopolassar; British Museum, London (BM 091104)",
  "full-title": "Nabopolassar's Inscriptions: 3",
  "full-info": "Long HTML description with full details"
}
```

Two missing fields: The feed currently lacks `id` (needed for navigation and bookmarking) and `author name` (needed for attribution). Before or during community bonding, I will submit a MR to the framework repo to add these fields.

- **Proposed Response (after MR):**


```
{
  "id": 782,
  "date": "2024-08-17",
  "thumbnail-url": "https://cdli.earth/pubs/daily_tablets/image_thumbnail.jpg",
  "url": "https://cdli.earth/pubs/daily_tablets/image.jpg",
  "blurb-title": "Nabopolassar's Inscriptions: 3",
  "theme": "Nabopolassar's Inscriptions",
  "blurb": "Clay barrel-cylinder with an inscription...",
  "full-title": "Nabopolassar's Inscriptions: 3",
  "full-info": "<p>Full article body with HTML...</p>",
  "author": "Wang Jinyan"
}
```

Adding new fields to the JSON response is fully backward-compatible, no existing consumers are affected. The Authors relationship already exists in the table model, so the backend change is minimal (adding contain(['Authors']) to the query and two fields to the JSON output).

The full dataset (?all=true) is approximately 1 MB, small enough to cache entirely on-device for offline access and instant search.

- **Database Schema(cdli_tablet):** Mobile app content comes from the cdli_tablet table, a curated collection of 500+ entries for now.

Column	Type	Purpose
id	int (PK)	Unique identifier
date_display	date	Publication/display date
theme	longtext	Category (e.g., "YBC", "Babylonian Slaves")
title_short	longtext	Entry title
title_long	longtext	Extended title - currently NULL for all entries
text_short	longtext	Short description / blurb
text_long	longtext	Full article body (HTML supported)
image_filename	varchar(255)	Image filename (served from /pubs/daily_tablets/)
author_id	int (FK)	Links to authors table

Note: Tablet entries are managed by admins via `/admin/cdli-tablet` (Role 1 only, session-based auth). **After the Role 8 MR**, designated tablet managers will also be able to manage entries without requiring full admin access. The admin panel is web-only and outside mobile app scope.

- **Image Serving:** Images are served as static files from https://cdli.earth/pubs/daily_tablets/. The feed provides two URLs per entry: a full-size image (url) and a thumbnail (thumbnail-url). The server checks for a thumbnail file and falls back to the full image if none exists.
- **Search:** The website implements server-side search via CakePHP's Search plugin, filtering only the theme column with SQL LIKE %keyword% plus an author dropdown with just 2 searchable fields.

The mobile app will implement client-side search over the cached dataset (~1 MB, all entries), filtering across all text fields: theme, title_short, description, and author name. This provides instant results and works offline, and covers more fields than the website.

- **Feedback:** The CDLI website uses **mailto:** links for all user communication.
 - cdli@ames.ox.ac.uk (general contact)
 - cdli-support@ames.ox.ac.uk (technical issues)

The mobile app follows the same approach using React Native's Linking API to open the device's email client, consistent with CDLI's existing workflow.

2. Highlights - JSON Endpoint (MR Required):

Endpoint: GET <https://cdli.earth/highlights>

- **Current State:**

The existing highlights page (/highlights) currently returns HTML only, which is not suitable for mobile consumption. Highlights are stored in the postings table, where different content types (**news, page, highlight, snippet, banner**) are distinguished using **posting_type_id**. highlight use type **3**.

The current controller already performs the required database queries, including published-status filtering and artifact image loading. The endpoint is **publicly accessible**, same as the daily tablet feed. The only missing capability is returning the data as structured JSON.

- **Proposed Change:**

Add a jsonSchema to the Posting entity and set the serialize view option in the highlights() controller action. When **/highlights.json** is requested, CakePHP's built-in JSON serialization will return structured JSON, following the same harmonized format used by Artifacts and other entities. This avoids creating new routes or duplicating query logic.

Example Response:

```
{
  "id": 187,
  "slug": "flood_tablet",
  "artifact_id": 273210,
  "created": "2021-10-01T00:00:00+00:00",
  "modified": "2022-08-22T08:53:14+00:00",
  "title": "The Flood Tablet",
  "body": "<p>A Mesopotamian tale of a great flood...</p>",
  "lang": "en",
  "created_by": 820,
  "modified_by": 1162,
  "publish_start": "2022-07-17T10:29:56+00:00",
  "artifact": {
    "id": 273210,
    "designation": "Gilgamesh II 124ff, J",
    "museum_no": "BM --",
    "accession_no": "K 03375 + Rm 0616",
    "created_by": 820,
    "artifact_assets": [
      {
        "id": 152860,
        "artifact_id": 273210,
        "asset_type": "photo",
        "file_format": "jpg",
        "path": "dl/photo/P273210.jpg",
        "thumbnail_path": "dl/tn_photo/P273210.jpg"
      }
    ]
  }
}
```

Results will be returned in descending publication order, handled by the existing ordering behavior in PostingsTable.

The table lists only the fields consumed by the mobile app. The JSON response includes additional fields following the harmonized format.

Field	Source	Purpose
id	postings.id	Navigation to highlight detail
title	postings.title	Card title and detail heading
body	postings.body	Preview text and full HTML content
artifact_id	postings.artifact_id	Link to artifact page on cdli.earth
path	artifact_assets (via artifact)	Detail view image
thumbnail_path	artifact_assets (via artifact)	Card image

- **Key Difference from Daily Tablet Feed:**

Daily tablet feed images are simple filenames served from /pubs/daily_tablets/. Highlight images are served through the Artifact → ArtifactAssets relationship. Each asset includes a path and thumbnail_path virtual property computed server-side. Public access filtering is handled by amendArtifactAssets(), which ensures only public images are included in the response.

Note: I have implemented and tested this endpoint locally. The MR will be submitted before or during the community bonding period, ensuring the endpoint is available before mobile development begins.

FRONTEND ARCHITECTURE & IMPLEMENTATION PLAN

Technology Stack:

Category	Library	Why
Framework	Expo SDK 55 (Dev Client)	Managed native config, dev builds with native modules
Navigation	Expo Router v6	File-based routing, automatic deep links, typed routes
Server State	TanStack Query v5	Fetch, cache, background refresh, offline persistence
Client State	Zustand v5	Bookmarks, preferences, UI state
Offline Storage	react-native-mmkv	30x faster than AsyncStorage, synchronous reads
Images	expo-image	Disk + memory cache, BlurHash placeholders, WebP/AVIF
Image Zoom	react-native-zoom-toolkit	Smooth pinch-to-zoom via Reanimated
List Rendering	FlashList (Shopify)	5-10x faster than FlatList
HTML Rendering	react-native-render-html	Renders HTML as native components, no WebView
Sharing	Built-in Share API	System share sheet, zero dependencies
Testing	Jest + RNTL	Unit and component testing
Build & Deploy	EAS Build + Update + Submit	Cloud builds for both platforms, OTA JS updates, store submissions
CI	GitHub Actions	Lint, type-check, and tests on every PR

Project Structure & Navigation:

Navigation is implemented using [Expo Router](#), which provides file based routing. Each file in the **app/** directory automatically becomes a screen, eliminating the need for manual route configuration.

Screens are organized using route groups that attach specific navigators: **(tabs)/** for bottom tab navigation, **(drawer)/** for drawer navigation, and top-level routes for standalone screens. This structure keeps navigation predictable and easy to maintain.

```
app/                                # Expo Router file-based routing
├── _layout.tsx                     # Root layout (drawer + tabs)
├── onboarding.tsx                 # First-launch onboarding
├── (tabs)/                         # Bottom tab navigator
│   ├── _layout.tsx               # Tab bar config (Home, Search, Saved, Settings)
│   ├── index.tsx                 # Home screen
│   ├── search.tsx               # Search + recently viewed + live suggestions + filter
│   ├── saved.tsx                # Bookmarks
│   └── settings.tsx             # Settings (clear history, clear cache, about)
├── (drawer)/                     # Drawer navigator
│   ├── _layout.tsx              # Drawer config + "Visit cdli.earth" link
│   ├── browse.tsx               # Browse All Tablets
│   ├── recently-viewed.tsx      # Recently Viewed
│   ├── about.tsx               # About CDLI
│   └── help.tsx                 # Help & FAQ + Contact Support (mailto:)
├── tablet/
│   └── [id].tsx                 # Tablet Detail (swipe, share, bookmark)
├── highlights.tsx                # All Highlights (linked from Home "View All")
└── search-results.tsx            # Full results

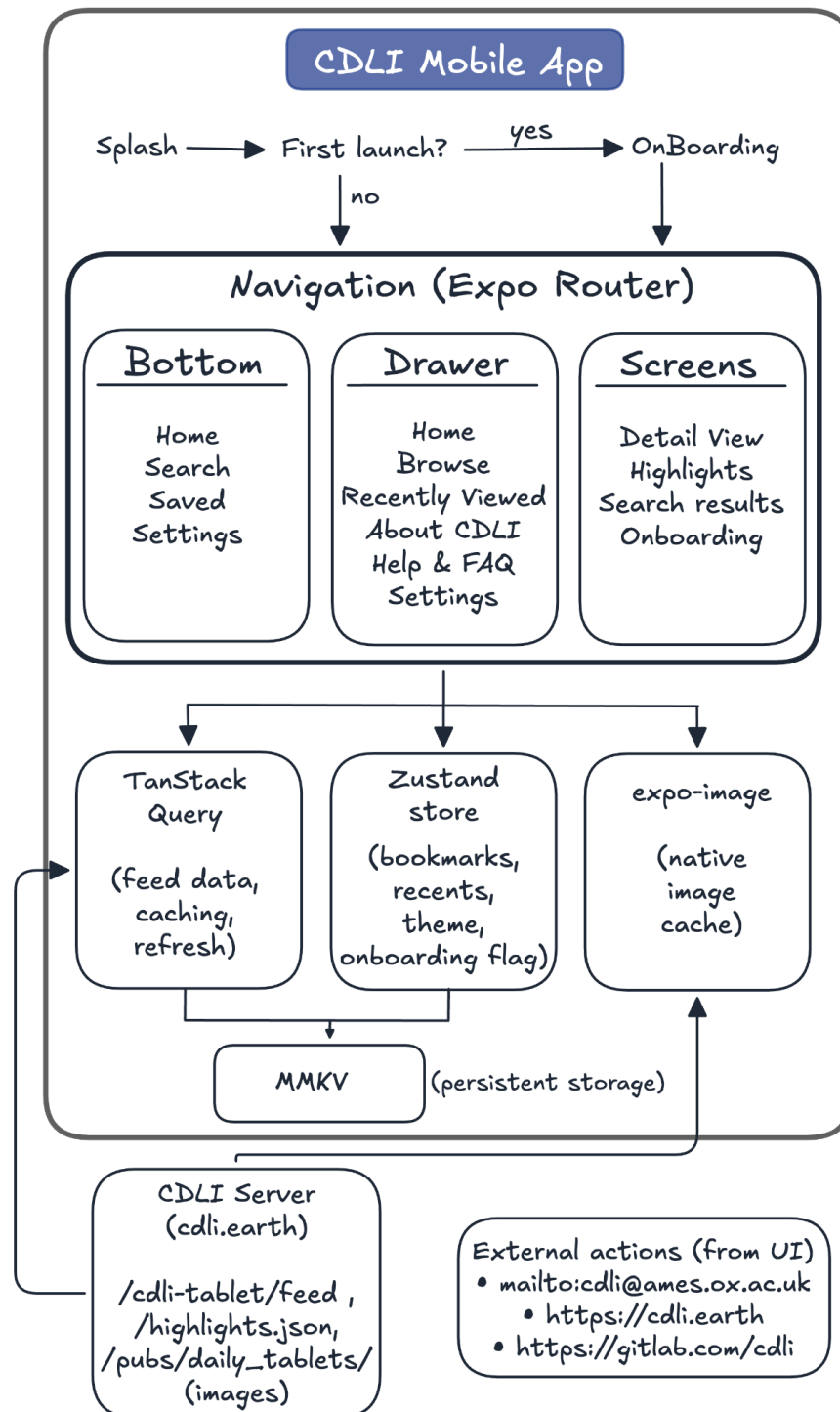
components/
├── TabletCard.tsx               # Tablet card (used in grid/list/search)
├── HeroCard.tsx                 # Tablet of the Day (Home)
├── CollectionCard.tsx           # Theme collection card (Home)
├── HighlightCard.tsx            # Highlight entry card
├── ZoomViewer.tsx               # Full-screen pinch-to-zoom modal
├── HTMLContent.tsx              # Converts API HTML to native components
└── OfflineBanner.tsx            # "You're offline" indicator

stores/
├── bookmarkStore.ts             # Saved tablet IDs
├── recentStore.ts               # Last 20 viewed tablet IDs
└── settingsStore.ts             # Onboarding flag, clear cache/history, theme

lib/
├── api.ts                      # API base URL, fetch functions
├── queryClient.ts               # TanStack Query + MMKV persister setup
├── storage.ts                   # MMKV instance setup
└── constants.ts                 # CDLI colors, font, URLs
```

Architecture Overview:

The following diagram illustrates the high-level architecture of the mobile application, showing how navigation, state management, caching layers, and external data sources interact.



The individual components and design decisions for each layer are explained in the sections that follow.

State Management:

The application separates state into three layers based on the type of data and persistence requirements.

Layer	Handles	Examples	Purpose
TanStack Query	Server data	Tablet feed, highlights	Fetching, caching, and background refresh
Zustand	Client state	Bookmarks, theme, recently viewed, onboarding flag	Lightweight state management without boilerplate
MMKV	Persistent storage	Data backing both layers	Fast synchronous storage that survives app restarts

Server data such as tablet entries and highlights requires fetching, caching, and background revalidation. TanStack Query manages this automatically.

Client-generated data such as bookmarks and recently viewed tablets does not originate from the API, making Zustand a simpler and more suitable solution.

MMKV acts as the shared persistence layer for both systems, ensuring cached data and user preferences remain available across app restarts.

Image caching is handled separately by expo-image, which maintains its own disk and memory cache for tablet images.

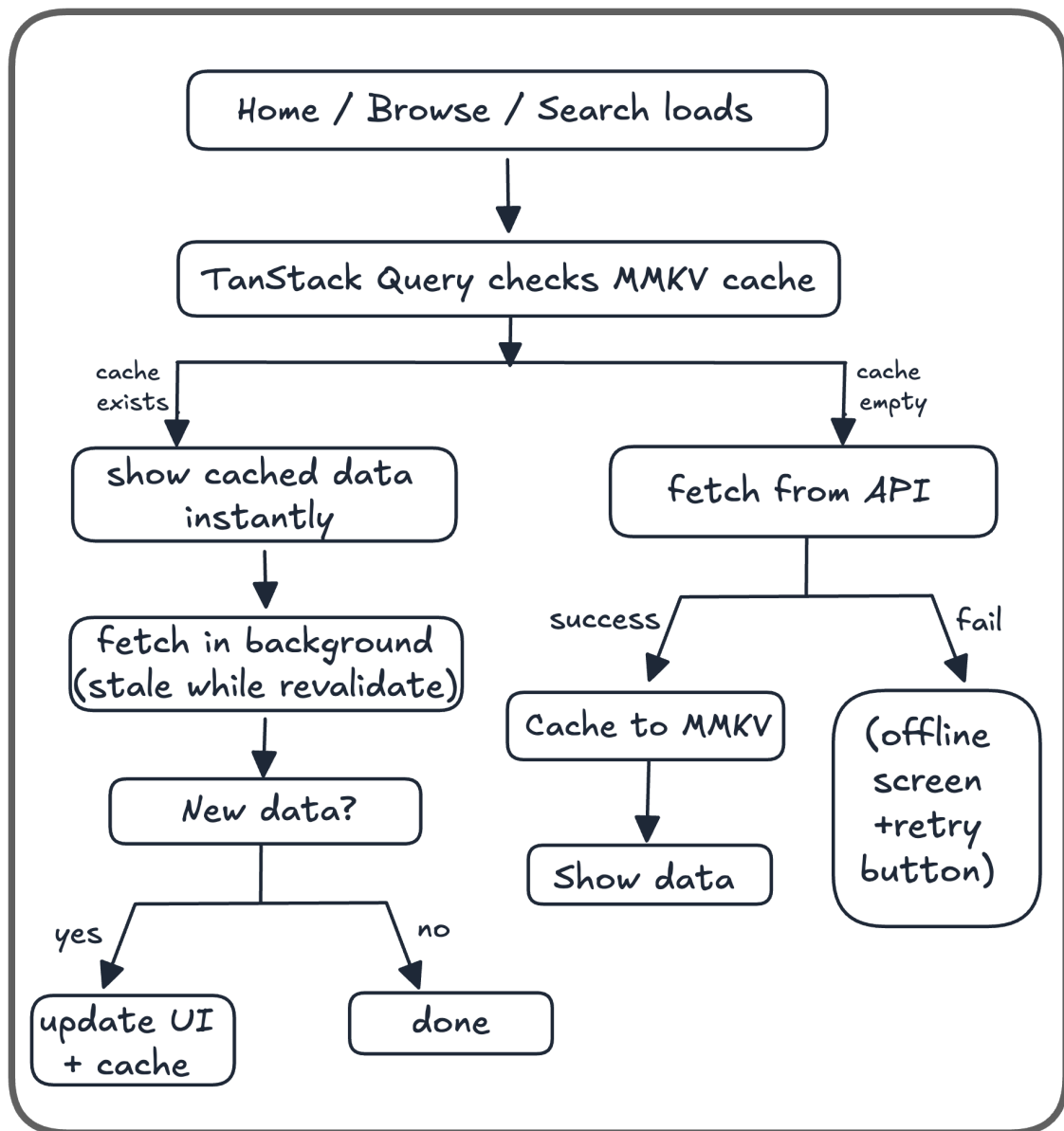
Offline Strategy:

The application is designed to remain functional without an internet connection. On first launch, the full tablet dataset (~1 MB) is fetched from the API and cached locally using TanStack Query backed by MMKV. After this initial fetch, users can browse, search, and view tablet entries entirely offline.

Data refreshing follows a stale-while-revalidate strategy. Cached data is displayed instantly when the app opens, while a background request checks for updates. If newer data is available, the cache and UI update automatically. If the

device is offline, the app continues serving cached data without interruption.

The following diagram illustrates the data flow and caching behavior used by the application.



Testing:

Testing will follow two approaches. **Jest and React Native Testing Library** will be used for unit and component tests during development. **Manual testing** will be performed across multiple Android and iOS devices to verify platform-specific behavior and edge cases.

I also have prior experience working with Jest and React Testing Library through open-source contributions, which closely relates to React Native Testing Library used in this project.

MOBILE APP - FEATURE BREAKDOWN

Complete Figma designs: [View all screens \(Light and Dark\)](#)

Core (Must Deliver)	Good to have
Home Screen Browse All Tablets Tablet Detail + Zoom Search & Filter Swipe Navigation Offline Caching Highlights Share Dark/Light Mode	Bookmarks Recently Viewed About CDLI Help & FAQ

1. Home Screen [Figma link](#)

The Home screen serves as the primary entry point of the application and provides users with quick access to the most relevant tablet content. The layout is structured into several sections designed to surface both curated and recent material.

- **Tablet of Day**

The home screen features a prominent hero card highlighting a tablet entry. Tapping opens the detail screen.

Daily tablet was a core feature of the original CDLI app, App Store reviews specifically praised it as a **fun treat to look forward to each day**.

- **Search [Figma link](#)**

A search bar positioned below the header acts as a navigation shortcut to the dedicated search screen. This keeps the home interface clean while still providing quick access to the search functionality.

- **Curated Collections**

This section presents horizontally scrollable collections grouped by theme, such as *"The Epic of Gilgamesh"*, *"Amarna Letters"*, *"Babylonian Slaves"*. Each

collection card displays a representative image and theme label. Selecting a collection filters the browsing interface to display tablets belonging to that theme.

- **Highlights**

The home screen features a dedicated Highlights section showcasing CDLI's most significant artifacts items like The Cyrus Cylinder, Codex Hammurapi, The Flood Tablet. These are curated by admins through the existing Postings system on the website.

2. Browse All Tablets [Figma link](#)

Browse screen displays all curated tablet entries available in the feed. Users can switch between grid view and list view, and sort the entries by:

- Newest first
- Oldest first
- Grouping by theme (e.g., “*Nabopolassar’s Inscriptions*”, “*Amarna Letters*”)

Efficient rendering is handled using **FlashList** ([Expo Documentation](#)), FlashList recycles off-screen views instead of repeatedly mounting and unmounting components, reducing memory usage and enabling smooth scrolling even on mid-range devices.

3. Swipe-Based Navigation

On the detail screen, users can swipe left or right to move between tablets without returning to the list. This interaction mirrors navigation patterns commonly used in gallery and news applications.

Swipe navigation was a **core interaction in the previous CDLI mobile** app. It reduces friction when browsing sequentially, whether sorted chronologically or grouped by theme.

4. Image Viewing [POC LINK](#) [VIDEO LINK](#)

Tablet images load in two stages: a lightweight thumbnail on the browse screen, and the full-resolution image on the detail screen with a **BlurHash placeholder** while loading.

Feature	Previous Apps	New React Native App
Pinch-to-zoom	Yes	Yes
Double-tap zoom	Partial	Yes
Panning	Yes	Yes (momentum-based)
Tap to close	No	Yes
Caching	Yes	Yes
Max zoom control	Limited	Yes (configurable)

Tapping the image opens a full-screen viewer with pinch-to-zoom, double-tap zoom, momentum-based panning and smooth interaction powered by Reanimated running on the native UI thread.

Note: During POC testing, I identified that some feed images are heavily compressed at the source. These can be replaced with optimized versions from the `/dl/photo/` endpoint via the admin panel for sharper zoom quality. Additionally, **proper validation for minimum file size and resolution** can be added to the upload form to prevent low-quality images in the future.

5. Search & Filter [Figma link](#)

Search screen provides client-side search across the full cached dataset. Users can search across **theme, title, description, and author name**, broader than the website, which only filters by theme and author. Search filters locally cached dataset using case-insensitive substring matching across four fields. The input is debounced to prevent filtering on every keystroke, and results appear instantly since the data is already cached locally.

A **Recently Viewed** section appears below the search bar, giving quick access to tablets the user has already explored.

6. Tablet Detail Screen [Figma link](#)

Displays the full scholarly content for a single tablet entry, including the high-resolution image (tap to open the zoom viewer), theme, author, date, title, short description, and full article body. Links within the content open in the system browser. Users can **swipe left or right** to navigate between tablets without returning to the list, enabling **faster exploration** of entries.

7. Share Tablet [Figma link](#)

Available on the detail screen and full image mode via a share icon, shares the tablet title, theme, and a link to CDLI using the system share sheet, users choose their preferred app (WhatsApp, email, etc.).

8. Bookmarks [Figma link](#)

Users can bookmark tablets to save them for later, accessible from the "Saved" tab in the bottom navigation bar. Bookmarks are stored locally on-device, no backend changes or user accounts needed.

9. Offline Access [Figma link](#)

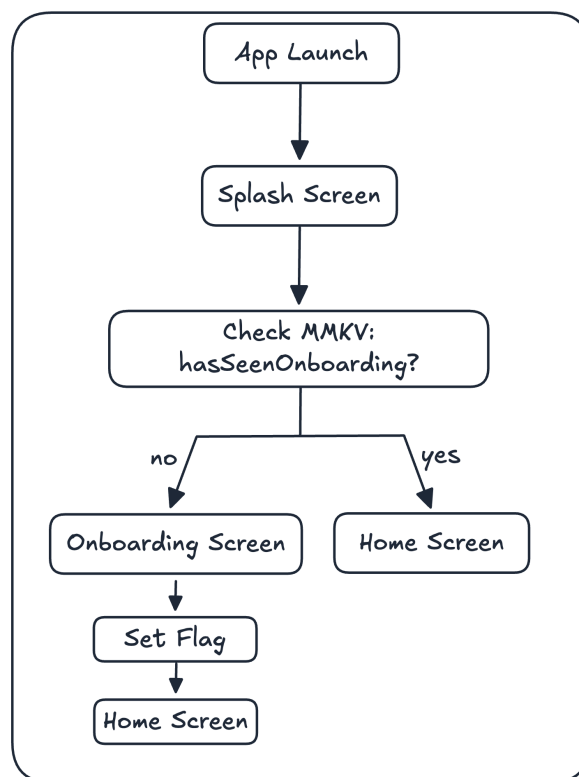
The full dataset (~1 MB) is cached locally after the first fetch. Users can browse, search, and view previously loaded tablets without an internet connection. Images are also cached on disk after first load.

UX Features

1. Splash Screen & Onboarding [Figma link](#)

Splash screen displays the CDLI logo and tagline on every app launch. On first launch only, a 3-screen swipeable onboarding introduces the app's core features: daily tablets, highlights, and search & share. Once completed, onboarding is not shown again.

The following diagram shows the app launch flow, including the first-launch onboarding check.



2. About CDLI [Figma link](#)

A static screen with CDLI's mission statement, team information, and links to cdli.earth and the CDLI GitLab repository. App version displayed at the bottom. All content is bundled, no network request needed.

3. Recently Viewed

Tracks the last 20 tablets the user has opened, accessible from the drawer. Displays the full entry with thumbnail, title, and theme. Users can clear their history from Settings.

4. Help & FAQ [Figma link](#)

A help screen with two sections: an FAQ accordion answering common questions about the app and CDLI's collection, and a "Contact Support" button using ***mailto:***, consistent with CDLI's existing email-based communication workflow. All FAQ content is bundled in the app.

5. Responsive Layouts [Figma link](#)

The app adapts to phones, tablets, and landscape orientation. Grid columns adjust automatically based on screen width, double column on phones, three/four on tablets. Layout recalculates on orientation change without restarting the app.

6. Dark Mode

The mobile app supports dark mode and follows the device's system appearance by default. Users can also manually switch between light and dark themes from the Settings screen.

PROJECT TIMELINE

Week	Dates	Tasks	Deliverable
-	May 1-24	Community Bonding: Meet mentors, finalize UI designs with feedback, submit Highlights JSON MR and Feed MR (add id + author), set up Expo project skeleton and EAS configuration	MRs submitted, project skeleton ready
1	May 25-31	Foundation: Navigation structure (Drawer + Bottom Tabs + Stack), CDLI design system (colors, typography, spacing), splash screen, onboarding flow, CI pipeline setup	App launches with navigation and theming
2	Jun 1-7	Home Screen: Layout, Tablet of the Day hero card, Curated Collections (horizontal scroll by theme), API integration with TanStack Query	Home screen functional with live data
3	Jun 8-14	Browse: Browse All Tablets with FlashList, grid/list view toggle, sorting (newest, oldest, group by theme)	Milestone 1: Data Visible
4	Jun 15-21	Detail + Image Viewer: Tablet Detail screen, HTML body rendering, full-screen image viewer with pinch-to-zoom and momentum-based panning	Complete detail experience with zoom
5	Jun 22-28	Search + Swipe: Client-side search with debounce, Recently Viewed section, swipe navigation between tablets on detail screen	Search and swipe working
6	Jun 29 - July 5	Offline + Buffer: Offline caching (TanStack persistence + MMKV), image caching (expo-image). Remaining days serve as a buffer for delays from Weeks 1–5	Milestone 2: Core App
7	Jul 6-10	Midterm Evaluation	Milestone 2 demonstrated to mentors
8	Jul 11-18	Highlights: Highlights section on Home screen, Highlights list and detail screens, integrate /highlights.json, image viewer with artifact deep-link	Highlights feature complete
9	Jul 19-25	Bookmarks + Share: Bookmark system (Zustand + MMKV), Saved tab in bottom navigation, share via system share sheet	Save and share working

10	Jul 26 - Aug 1	UX Features: About CDLI, Help & FAQ, Light mode, Settings screen (clear history, theme toggle)	All secondary features complete
11	Aug 2-8	Responsive + Buffer: Responsive layouts (phone, tablet, landscape), adaptive grid columns. Remaining days serve as buffer for delays from Weeks 7-9	Milestone 3: Feature Complete
12	Aug 9-15	Testing: Unit tests (Jest + RNTL), integration tests, performance testing across available devices, bug fixes.	App stable and tested
13	Aug 16-24	Ship: App Store + Play Store submission via EAS Submit, final documentation, GSoC final report	Milestone 4: Shipped

The project is structured around **four milestones** aligned with the GSoC evaluation checkpoints. Each milestone produces a testable deliverable, allowing mentors to evaluate working software at every stage.

Development during **Weeks 1-10** focuses on **feature implementation**. **Week 12** is dedicated to **testing, bug fixes, and performance validation** across available devices.

Two buffer windows are built into the schedule - one before the midterm evaluation and one before the final phase - to absorb unexpected delays.

Availability:

I will dedicate **35-40 hours per week** throughout the project. During **June 21-29**, I have university **exams** and my availability will temporarily reduce to approximately **2 hours per day**. To account for this, Week 5 is scoped conservatively and Week 6 includes buffer days to absorb any spillover before the midterm evaluation.

From **June 30 to August 3**, I will be on **university vacation** with no classes or exams. During this period, I will dedicate extended hours to development.

If the project requires additional time, I will communicate with my mentors early to discuss adjustments to the timeline.

I have no other commitments during this period. If selected for GSoC, this project will be **my primary focus for the entire summer**.

Buffer Strategy:

Weeks 6 and 11 serve as buffer windows. The primary tasks planned for these weeks (offline caching and responsive layouts) are scoped to 3-4 days, leaving 2-3 buffer days.

If earlier weeks run on schedule, this time will be used for **bug fixes and incorporating mentor feedback**. If delays occur, the buffer absorbs them without affecting milestone deadlines.

Milestones

Milestone 1: Data Visible - Week 3

The application fetches real data from the CDLI feed and displays it. The Home screen shows the Tablet of the Day and curated collections, while the Browse screen lists all tablets with grid/list toggle and sorting. Mentors can verify that the app works with live content.

Milestone 2: Core App - Midterm (July 10)

The complete primary user flow functions end-to-end: browsing tablets, opening details, zooming into images, searching the dataset, and swiping between entries - all working offline.

Milestone 3: Feature Complete - Week 11 (August 8)

All planned features are implemented, including Highlights, bookmarks, sharing, dark mode, responsive layouts, and supporting UX screens. The application is feature-frozen after this milestone.

Milestone 4: Shipped - Final (August 24)

The application is tested, stable, and submitted to both app stores. The GSoC final report is completed and the codebase is documented for future contributors.

EXPECTATIONS FROM MENTORS

I will aim to work independently and keep mentors updated through clear merge requests, documentation, and regular communication. Guidance in a few areas would be particularly helpful.

During the community bonding period, the mobile app will require a **new repository** under the CDLI GitLab organization. Mentor guidance on repository naming and initial setup will help ensure the project follows the organization's conventions.

I will also share **Figma designs and the proposed navigation** structure early in the bonding period so mentors can provide feedback before development begins, allowing UI decisions to be aligned and avoiding rework later.

Two backend merge requests - the **Highlights JSON endpoint** and the **Feed field additions** - will be submitted before or during community bonding. Early review will ensure the mobile application can depend on these endpoints from the start of development.

Throughout development, I will **open merge requests** as features are completed. A review turnaround of roughly one week would allow feedback to be incorporated while the next week's work proceeds in parallel. I will also prepare comprehensive project documentation covering app architecture, development setup, API integration, and contribution guidelines for future contributors.

To keep progress transparent, I will maintain a **shared tracking board/sheet** showing completed, in-progress, and upcoming work. Merge requests will remain small and clearly structured so review time can be used efficiently.

The features listed in this proposal are based on user research, survey data, and the project description. If mentors or the community identify additional needs during development, I am open to adjusting priorities and incorporating improvements where appropriate.

CONTRIBUTIONS SO FAR

[Total MRs: 15+](#) (15 Merged, 0 Open) | [Issues Opened: 5+](#)

1. DevOps / Infrastructure

3 MRs

#	MR	Commit Message	Priority	Status
1	!1105	Devops migrate from node-sass to Dart Sass in Docker	-	Merged
2	!1117	fix shell syntax errors and deprecated mysql command in setup.sh	-	Merged
3	!1118	add cloudflare turnstile test keys for local development	-	Merged

2. Feature / Enhancement

3 MRs

#	MR	Commit Message	Priority	Status
1	!1112	added non-public indicator chips for artifacts, inscriptions, and images	P1	Merged
2	!1111	extend museum, accession, and excavation number field lengths	P1	Merged
3	!1103	Hide crowdsourcing button if user already has privilege	-	Merged

3. Search / Backend / Logic

2 MRs

#	MR	Commit Message	Priority	Status
1	!1109	trim whitespace from search query values	-	Merged
2	!1110	fix redirect after deleting user to return to previous page	-	Merged

4. Frontend

7 MRs

#	MR	Commit Message	Priority	Status
1	!1106	add mobile footer links for terms of use, privacy policy and imprint	P2	Merged

2	!1107	replace CURSOR_URL placeholder with cursor:pointer	P3	Merged
3	!1104	fix hamburger menu overlapped by carousel controls	-	Merged
4	!1108	fix mismatched HTML tags in articles view	-	Merged
5	!1102	Set nicer page title on 404 error pages	-	Merged
6	!1136	fix long link overflow in cdli-table cells	-	Merged
7	!1137	fix long inscription edit and broken tab switching	-	Merged

5. Issues Created

5 Issues

#	MR	Commit Message	Status
1	#2503	add cloudflare turnstile test keys to .env.examples for local development	Done
2	#2502	mysql: command not found during Stage-3 database setup	Done
3	#2505	shell syntax errors in setup.sh produce misleading error messages	Done
4	#2506	stage-3 database download via GitLab API always fails	Open
5	#2532	docs: fix outdated Logstash schedule and add ElasticSearch/Logstash setup reference	Open

WHY ME?

Since joining CDLI in early 2026, I have been actively contributing to the project and helping other contributors in the community. I am currently helping moderate the Slack channel, where I help contributors set up the framework locally by guiding them through Docker setup, database configuration, and environment issues. I have also started reviewing pull requests, helping ensure changes are correct and aligned with the project.

While working with the framework, I identified some problems in the setup scripts that made installation difficult for new contributors. I reported these issues and submitted fixes so that future contributors can set up the project more easily. Improving the onboarding experience also helped me gain a deeper understanding of how the framework is structured and how its components interact.

For this proposal, I spent time studying the CDLI codebase beyond the parts directly related to the mobile app. I explored the CakePHP backend, the `cdli_tablet` feed construction, and the Postings system used for highlights. The architecture and API changes proposed in this project are based on this analysis of the existing system.

I care about contributing to open-source communities and helping other contributors succeed. This project is an opportunity to deliver a maintainable mobile application that integrates cleanly with CDLI's infrastructure and continues to benefit the project beyond the GSoC program.

FUTURE WORK

I have been contributing to open source for the past few months, and I genuinely enjoy it, it has gradually become a part of my life. Although I joined the CDLI project only recently, the work here aligns closely with my interests, and I really enjoy contributing and interacting with the community.

Future improvements could include multilingual support, push notifications for the Tablet of the Day, a home screen widget, note-taking on individual tablets, and additional content types from the website such as news articles.

I would like to stay connected with the CDLI community in the future and continue maintaining the mobile application beyond the GSoC period. CDLI has experimented with mobile apps in the past, but keeping them active over time has been challenging, and I would love to help change that.

Use of AI Tools

I use AI tools as part of my workflow for learning and exploration.

For this proposal, I used ChatGPT to improve English grammar and phrasing, as English is not my first language, and to refine the structure of sections. I also used tools like Perplexity for researching library options and comparing approaches. I also use AI tools to learn new concepts, topics, and to understand unfamiliar things. I plan to continue using them in the same way during GSoC, as a learning and research aid.

THANK YOU